

Task:

1. Crawl the text reviews of Nangman BBQ restaurant on the provided website
2. Cluster the crawled restaurants into 3 groups based on their reviews
3. Identify the characteristics of each sorted groups

Skills: Web Scraping, Data Preprocessing, Data Clustering, Data Visualization, Data Interpretation

1. Crawl all the text reviews of the Nangman BBQ restaurant on Opentable.com:

```
[1]: from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```
[3]: driver = webdriver.Chrome()
```

```
[5]: all_reviews = []
for page in range(1, 17):
    url = f"https://www.opentable.com/r/nangman-bbq-new-york?page={page}"
    driver.get(url)
    time.sleep(3)

    reviews = driver.find_elements(By.CLASS_NAME, "_6rFG6U7PA6M-")
    for el in reviews:
        all_reviews.append(el.text)
```

```
[7]: all_reviews
```

```
[7]: ['This was my second visit, and it was just as outstanding as the first. The BBQ was flavorful, tender, and grilled to perfection. What stood out even more this time was the variety of Korean side dishes that came with the meal – each one was fresh, well-prepared, and complemented the meat beautifully. The service was warm and attentive, striking just the right balance without being overbearing. You can tell that every detail, from the food to the hospitality, is handled with care. I'll definitely be returning – it's the kind of place you find yourself craving again and again.\nRead more',
      'Truly a hidden gem!\n\nThe meats were perfectly grilled, the side dishes were fresh and tasty, and the service was warm and thoughtful. One of the highlights was that they grill the meat for you right at the table—it made everything so relaxing and enjoyable. The atmosphere was cozy and welcoming—just the kind of place you want to keep coming back to.\n\nHighly recommend. I truly hope this place gets all the love it deserves!\nRead more',
      'Best BBQ experience of my entire life. I never write reviews, but I had to for Nangman. The quality and service here is better than any All-You-Can-Eat restaurant, and the portions are perfect, so you're left with a satisfied and full stomach and no need to worry about wasting food. Would 100% recommend for people who love great food and specially catered service!\nRead more',
      'Food mediocre to poor. Service nice but slow. Way over priced given quality.',
      "I came here for a birthday dinner on a Wednesday night with a group of 9 people. This place was not in K-Town so it's not crowded which was great. We ordered the Onggijonggi which originally is for 5-6 people but we were able to do it for 9 people. It was a good deal since we were able to try a lot of d
```

2. Convert the text reviews to a bag-of-words using TF-IDF scores; Convert upper case letters to lower case, delete stopwords, and lemmatize the words

```
[10... from nltk.tokenize import word_tokenize
```

```
[12... from nltk.corpus import stopwords
```

```
[14... nltk_stopwords= set(stopwords.words("english"))
nltk_stopwords
```

```
[14... {'a',
      'about',
```

```

[18... from nltk.stem import WordNetLemmatizer
      lem = WordNetLemmatizer()

[20... def my_tokenizer(r):
      r1 = word_tokenize(r) #convert into vector? tokenize it
      r2 = [w.lower() for w in r1 if w.isalpha()] #convert the upper into lowercase
      r3 = [w for w in r2 if not w in nltk_stopwords] #remove stopwords
      r4 = [lem.lemmatize(w) for w in r3]
      return r4

[22... from sklearn.feature_extraction.text import TfidfVectorizer

[24... tfidf = TfidfVectorizer(tokenizer = my_tokenizer, token_pattern = None)

[26... tfidf_review = tfidf.fit_transform(all_reviews)
      tfidf_review

[26... <160x1041 sparse matrix of type '<class 'numpy.float64'>'
      with 3194 stored elements in Compressed Sparse Row format>

[27... tfidf_review.toarray()

[27... array([[0.         , 0.         , 0.         , ..., 0.         , 0.         ,
          0.         ],
        [0.         , 0.         , 0.         , ..., 0.         , 0.         ,
          0.         ],
        [0.         , 0.         , 0.         , ..., 0.         , 0.         ,
          0.         ],
        ...,
        [0.         , 0.         , 0.         , ..., 0.         , 0.         ,
          0.         ],
        [0.         , 0.         , 0.         , ..., 0.         , 0.24533373,
          0.         ],
        [0.         , 0.         , 0.         , ..., 0.         , 0.         ,
          0.         ]])

[30... len(tfidf_review.toarray())

[30... 160

[32... tfidf_terms = tfidf.get_feature_names_out()
      tfidf_terms

[32... array(['able', 'abroad', 'absolutely', ..., 'yummy', 'è', '떡갈비'],
      dtype=object)

[34... len(tfidf_review.toarray())

[34... 160

[36... import pandas as pd

[38... review_bow = pd.DataFrame(tfidf_review.toarray(), columns = tfidf_terms)

[40... review_bow

```

	able	abroad	absolutely	accommodating	accompaniment	accompanying	acknowledged	acknowledg
--	------	--------	------------	---------------	---------------	--------------	--------------	------------

0	0.000000	0.0	0.0	0.000000	0.0	0.0	0.0	0.0
1	0.000000	0.0	0.0	0.000000	0.0	0.0	0.0	0.0
2	0.000000	0.0	0.0	0.000000	0.0	0.0	0.0	0.0
3	0.000000	0.0	0.0	0.000000	0.0	0.0	0.0	0.0
4	0.216287	0.0	0.0	0.000000	0.0	0.0	0.0	0.0

3. Choose the "optimal" number of clusters using the elbow method, and apply K-means clustering on the text reviews

```
[43... from sklearn.cluster import KMeans
```

```
[45... km = KMeans(n_clusters=3, random_state = 0)
```

```
[47... km_model = km.fit(tfidf_review)
```

```
[49... clusters = km_model.labels_.tolist()  
clusters
```

```
0,  
2,  
0,  
0,  
2,  
2,  
2,  
0,  
1,  
0,  
0,  
2,  
0,  
2,  
0,  
2,  
1,  
2,  
2,  
2,
```

```
[51... len(clusters)
```

```
[51... 160
```

```
[53... km_model.inertia_
```

```
[53... 147.01995386561313
```

```
[55... Ks=list(range(1,6))  
Ks
```

```
[55... [1, 2, 3, 4, 5]
```

```
[57... distance = []
```

```
for k in Ks:  
    km_n= KMeans(n_clusters=k, random_state=0)  
    km_model_n=km_n.fit(tfidf_review)  
    d = km_model_n.inertia_  
    distance.append(d)
```

```
#the first loop the sum of each documents
```

```
[59... distance
```

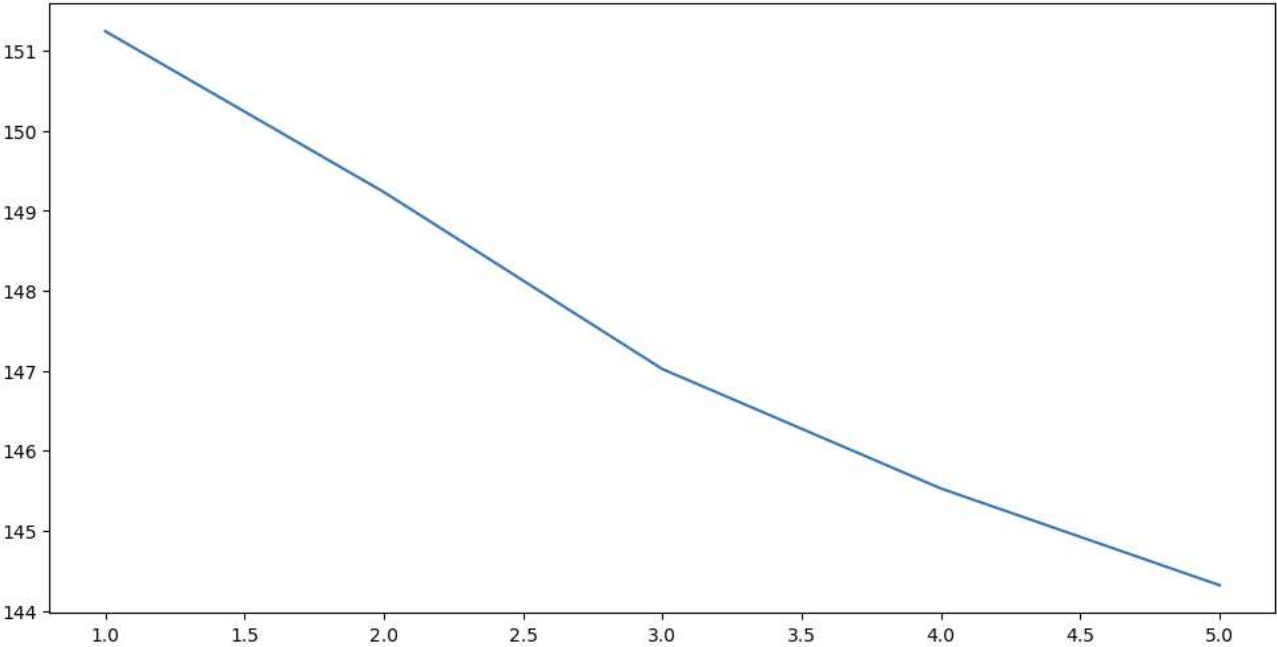
```
[59... [151.24414503250733,  
149.23313473821037,  
147.01995386561313,  
145.52688671440796,  
144.31784461636414]
```

```
[61... from matplotlib import pyplot as plt
```

```
[62... plt.figure(figsize=(12,6))  
plt.plot(Ks, distance)  
plt.show()
```

```
[61... from matplotlib import pyplot as plt
```

```
[62... plt.figure(figsize=(12,6))
plt.plot(Ks, distance)
plt.show()
```



4.Create a table that shows text reviews and the cluster number that each text review belongs to, as shown below:

```
[66... import pandas as pd
```

```
[68... table= pd.DataFrame({"Review": all_reviews, "Cluster": clusters})
table
```

	Review	Cluster
0	This was my second visit, and it was just as o...	2
1	Truly a hidden gem!\n\nThe meats were perfectl...	2
2	Best BBQ experience of my entire life. I never...	2
3	Food mediocre to poor. Service nice but slow...	0
4	I came here for a birthday dinner on a Wednesd...	0
...	...	...
155	Nangman has a very cute outdoor set up (indoor...	2
156	A disappointing experience, in every sense. We...	2
157	We had an outstanding dinner here! Came on a w...	2
158	Veramente un posto interessante da visitare co...	1
159	Its new so they just need to work out the serv...	1

160 rows x 2 columns

5. Characterize each cluster using the most common/relevant words for each cluster;

```
[71...] km_model.cluster_centers_  
[71...] array([[0.00636139, 0.          , 0.01108098, ..., 0.          , 0.          ,  
          0.          ],  
          [0.          , 0.00618465, 0.03060498, ..., 0.          , 0.00663064,  
          0.          ],  
          [0.00478961, 0.          , 0.00609616, ..., 0.00242348, 0.          ,  
          0.00114818]])  
[73...] cent = km_model.cluster_centers_.argsort()[:, :-1]  
cent  
[73...] array([[ 799,  388,  350, ...,  533,  531, 1040],  
          [ 297,  291,  350, ...,  642,  641,    0],  
          [ 394,  350,   78, ...,  485,  141,  395]])  
[75...] tfidf_terms[7]  
[75...] 'acknowledgement'  
[77...] for i in range(0,3):  
        top_words = [tfidf_terms[x] for x in cent[i, :5]]  
        print(top_words)  
['service', 'good', 'food', 'great', 'quality']  
['excellent', 'everything', 'food', 'service', 'delicious']  
['great', 'food', 'bbq', 'experience', 'korean']
```

Conclusion

1. Reviews in cluster 0 highlight the overall satisfaction with the restaurant
2. Reviews in cluster 1 highlight the good food and service
3. Reviews in cluster 2 highlight specific dishes such as bbq korean

Thank you